

Real-Time Process Mining for Quick-Commerce Order Fulfillment & Dark-Store Dispatch

A Kafka → PySpark Structured Streaming → Delta Lake case study applying real-time BPM process mining, SLA monitoring, and anomaly detection to a simulated dark-store fulfillment process.

Name: _____

Register No: _____

Class / Section: _____

Project repository: github.com/itzbhav/realtime-lakehouse-pipeline (folder: bpm/)

1. Business Context

A quick-commerce dark store (Blinkit / Instamart style) operates under a strict 15-minute delivery promise. An order moves through picking, bagging, rider dispatch, and doorstep delivery; a delay or process failure at any single stage — a stuck picker, an undocumented item substitution, a rider handoff that never happens — directly breaks the SLA and costs the business a refund or a lost customer. This case study builds the real-time pipeline that would catch those failures **as they happen**, not in a post-mortem the next morning.

The pipeline reuses the same Docker-based lakehouse infrastructure (Kafka, Spark Structured Streaming, Delta Lake on MinIO, Hive Metastore) built for the Data Engineering Lab project, applied here to a new BPM-shaped event domain corresponding to Scenario 4 (Superstore / Quick-Commerce) of the assignment brief.

2. Process Stages

The order fulfillment process consists of six sequential stages, one exception activity, and one alternate terminal (cancellation) path:

#	Activity	Swimlane	Description
0	ORDER_RECEIVED	Customer App	Customer places the order; case begins
1	PICKER_ASSIGNED	Dark Store Ops	A store picker is assigned to fulfil the order
—	ITEM_SUBSTITUTED	Dark Store Ops (exception)	Out-of-stock item replaced; does not advance the main sequence
2	ITEMS_BAGGED	Dark Store Ops	All items packed and ready for handoff
3	RIDER_ASSIGNED	Rider / Courier	A delivery rider is assigned (picker→rider handoff)
4	DISPATCHED_FROM_DARKSTORE	Rider / Courier	Rider leaves the dark store with the order
5	DOORSTEP_DELIVERED	Rider / Courier	Order handed to the customer (rider→doorstep handoff)
—	ORDER_CANCELLED	Exception Handling	Alternate terminal: picker/rider unavailable or timeout

3. Event Taxonomy

Every process event follows the case_id / activity / timestamp taxonomy required by the assignment, with resource, location, and a metadata payload:

```
{
  "case_id": "ORD-SUPER-000047",
  "activity": "ITEMS_BAGGED",
  "timestamp": "2026-09-28T11:03:40Z",
  "resource": "Picker_Emp_112",
  "location": "DarkStore_Coimbatore_North",
  "metadata": {
    "item_count": 8,
    "target_total_minutes": 15,
    "picking_sla_minutes": 3
  }
}
```

ITEM_SUBSTITUTED events carry two additional metadata fields (substituted_item, reason) and are logged as exception events that do not advance the case's main sequence position.

4. BPMN Process Map

BPMN Process Map — Quick-Commerce Order Fulfillment & Dark-Store Dispatch

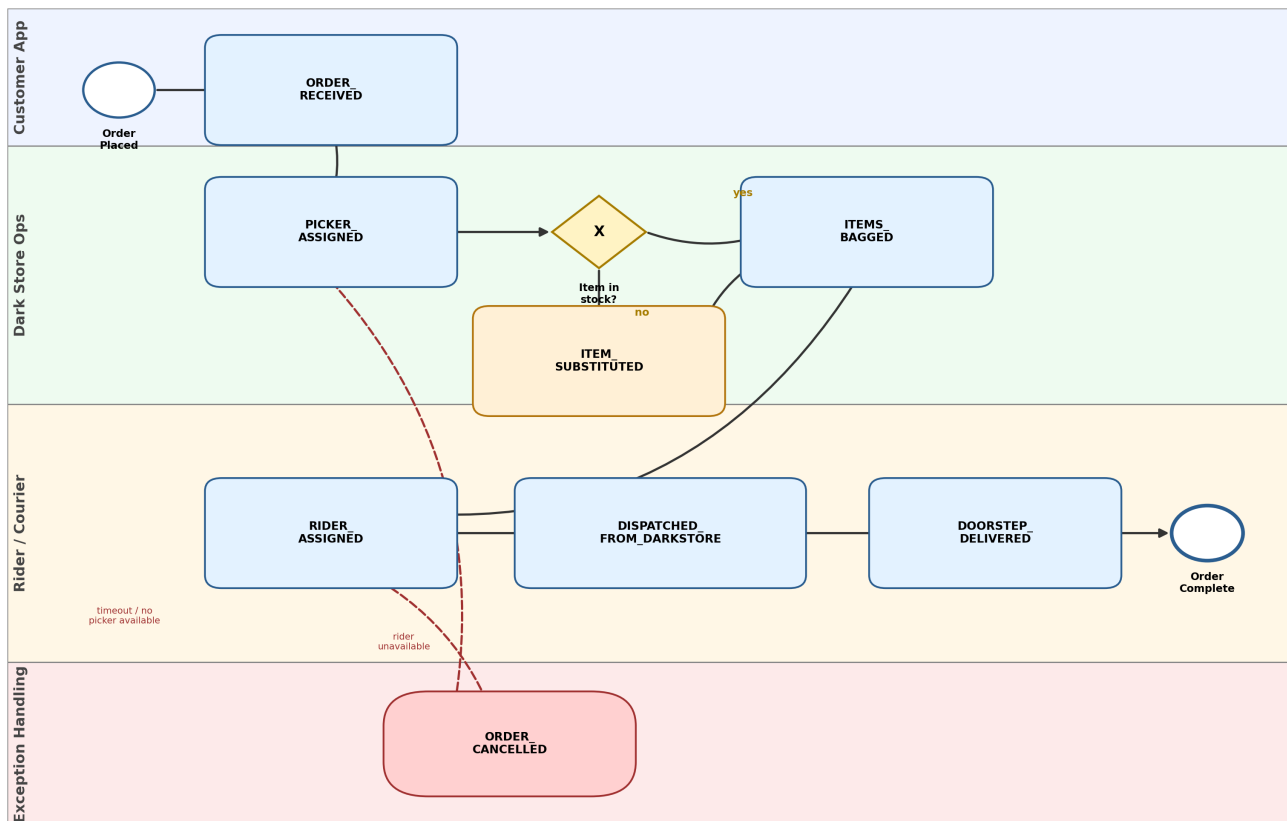


Figure 1. Three swimlanes (Customer App / Dark Store Ops / Rider), one exclusive gateway for the out-of-stock substitution exception, two courier-handoff points (picker→rider at ITEMS_BAGGED→RIDER_ASSIGNED, rider→doorstep at DISPATCHED_FROM_DARKSTORE→DOORSTEP_DELIVERED), and the cancellation escape path.

5. Key Performance Indicators

KPI	Definition	Computed in
Cycle time	Minutes between consecutive stage events (stage_minutes); total ORDER_RECEIVED→DOORSTEP_DELIVERED time (total_minutes)	bpm_process_engine.py, per event
SLA limits	Per-stage targets: 1 / 3 / 2 / 2 / 7 min, summing to the 15-min target_total_minutes promise. Picking stage is data-driven from metadata.picking_sla_minutes; others are fixed ops constants.	STAGE_SLA_MINUTES constant
Bottleneck stage	The stage-to-stage transition with the highest average stage_minutes or highest SLA-breach rate	Printed every batch; queryable via bpm.process_events
Throughput	Count of DOORSTEP_DELIVERED events per micro-batch / time window	Printed as part of the batch KPI line
Drop-off rate	% of cases whose latest known state (bpm.case_state) never reached DOORSTEP_DELIVERED	Query against case_state

6. Real-Time Process Analytics & Anomaly Detection Engine

(Rubric component 2 — 15 marks)

`bpm_process_engine.py` is a PySpark Structured Streaming job that reads `order_fulfillment_events` from Kafka and, per micro-batch, reconstructs each case's process state: first via an in-batch window function (since a fast-moving case can have several events land in the same 10-second trigger), falling back to a persisted `case_state` Delta table — upserted via a Delta MERGE — for state carried across batches. Every event, anomalous or not, is written to `bpm.process_events` as an immutable audit log with the flags below attached, so nothing is silently dropped.

6.1 Anomaly Detection Rules

Anomaly	Detection rule
SLA breach	<code>stage_minutes > sla_limit_minutes</code> for that transition, or (at delivery) <code>total_minutes > metadata.target_total_minutes</code>
Duplicate task	Incoming activity equals the case's previous activity
Out-of-sequence activity	Incoming stage index $\leq$ the case's previous stage index (went backward or repeated out of place), or the case's very first observed event is not <code>ORDER_RECEIVED</code>
Process deviation	Surfaces as a combination of the above — e.g. <code>DOORSTEP_DELIVERED</code> with no prior <code>DISPATCHED_FROM_DARKSTORE</code> shows up as an out-of-sequence flag on the delivery event

6.2 Verified Run — Injected vs. Detected Anomalies

The producer (`bpm_producer.py`) simulated 40 orders, deliberately injecting anomalies at fixed rates, then `bpm_process_engine.py` was run against the resulting stream. Detected counts are compared against what the producer actually injected:

Anomaly type	Injected by producer	Detected by engine	Match
SLA breach (cases)	4	4 cases $\times$ 2 flagged rows each (stage + cascading total) = 8	✓ exact
Duplicate task	2	2	✓ exact
Out-of-sequence	1	1	✓ exact
Substitution	7	7	✓ exact
Drop-off (never delivered)	1	1 (stuck at <code>RIDER_ASSIGNED</code> in <code>case_state</code>)	✓ exact
Total events processed	—	247	—
Orders delivered (throughput)	39 / 40	39	✓ exact

Console output from the live run:

```
>>> RUNNING TOTALS — events: 247 | delivered: 39 | sla_breaches: 8 | duplicates: 2 |  
    out_of_sequence: 1 | substitutions: 7
```

6.3 SLA-Breach Detail (queried from `bpm.process_events`)

All four deliberately-breached cases show exactly the expected pattern: the specific stage they were designed to blow, plus a cascading total-time breach at delivery (realistic — a late stage naturally makes the whole order late):

case_id	activity	stage_min	sla_limit	sla_breached	total_min	total_breached
ORD-SUPER-000001	PICKER_ASSIGNED	3.47	1.0	true	3.47	false
ORD-SUPER-000001	DOORSTEP_DELIVERED	6.92	7.0	false	16.97	true
ORD-SUPER-000003	PICKER_ASSIGNED	3.93	1.0	true	3.93	false
ORD-SUPER-000003	DOORSTEP_DELIVERED	6.92	7.0	false	17.27	true

case_id	activity	stage_min	sla_limit	sla_breached	total_min	total_breached
ORD-SUPER-000019	ITEMS_BAGGED	6.40	3.0	true	7.33	false
ORD-SUPER-000019	DOORSTEP_DELIVERED	6.73	7.0	false	17.82	true
ORD-SUPER-000030	RIDER_ASSIGNED	5.07	2.0	true	8.73	false
ORD-SUPER-000030	DOORSTEP_DELIVERED	6.90	7.0	false	17.33	true

6.4 Duplicate & Out-of-Sequence Detail

case_id	activity	prev_activity	is_duplicate	is_out_of_sequence
ORD-SUPER-000007	PICKER_ASSIGNED	ITEMS_BAGGED	false	true
ORD-SUPER-000017	DISPATCHED_FROM_DARKSTORE	DISPATCHED_FROM_DARKSTORE	true	false
ORD-SUPER-000021	DISPATCHED_FROM_DARKSTORE	DISPATCHED_FROM_DARKSTORE	true	false

Case 000007's out-of-sequence swap is visible one level deeper too: its PICKER_ASSIGNED and ITEMS_BAGGED events had their timestamps deliberately swapped, so the engine correctly sees ITEMS_BAGGED arriving (by event time) before PICKER_ASSIGNED for that case — exactly the process-order violation the detector is designed to catch.

6.5 Stage Duration / Bottleneck Analysis

Real output from the live run (average minutes per stage transition):

prev_activity	activity	avg_minutes	n
DISPATCHED_FROM_DARKSTORE	DOORSTEP_DELIVERED	6.83	39
PICKER_ASSIGNED	ITEMS_BAGGED	2.91	39
ITEMS_BAGGED	RIDER_ASSIGNED	1.92	39
RIDER_ASSIGNED	DISPATCHED_FROM_DARKSTORE	1.85	39
ORDER_RECEIVED	PICKER_ASSIGNED	0.98	39
ITEMS_BAGGED	PICKER_ASSIGNED *	2.80	1
PICKER_ASSIGNED	RIDER_ASSIGNED *	1.87	1
ORDER_RECEIVED	ITEMS_BAGGED *	0.87	1
DISPATCHED_FROM_DARKSTORE	DISPATCHED_FROM_DARKSTORE *	0.29	2

* rows are direct evidence of injected anomalies: the out-of-sequence case (000007) disturbs three neighbouring transition pairs in the audit log (its own swap plus both adjacent pairings), and the two duplicate DISPATCHED_FROM_DARKSTORE events show up as a same-activity transition with a near-zero gap.

Bottleneck identified: DISPATCHED_FROM_DARKSTORE → DOORSTEP_DELIVERED, at 6.83 minutes average against a 7-minute SLA — the transit leg consumes the largest share of the 15-minute delivery promise and is the stage with the least slack.

7. Querying the Lakehouse for Process Metrics

(Rubric component 3 — 15 marks)

7.1 Working today — via spark-sql

Both `bpm.process_events` (the immutable fact log) and `bpm.case_state` (latest per-case snapshot) are registered in Hive Metastore and queryable by name today:

```
SELECT last_activity, count(*)
FROM bpm.case_state
GROUP BY last_activity
ORDER BY 2 DESC;
```

```
-- Verified result:
-- DOORSTEP_DELIVERED    39
-- RIDER_ASSIGNED         1
```

This is the “active bottleneck / drop-off” query the assignment asks for: it shows exactly where in-flight and abandoned orders currently sit, with the one drop-off case correctly stuck at `RIDER_ASSIGNED`, matching the producer's own injected drop-off exactly.

7.2 Querying via Trino (pending Week 3 infrastructure)

Trino is not yet in the project's `docker-compose.yml` — that is a separate, later phase of the Data Engineering Lab project. Once added with a catalog pointing at this same Hive Metastore, the following queries are ready to run for this rubric section, written against the live schema above:

```
-- Order Turnaround Time (TAT) per dark store
SELECT location,
       round(avg(total_minutes), 2) AS avg_tat_minutes,
       round(approx_percentile(total_minutes, 0.95), 2) AS p95_tat_minutes
FROM bpm.process_events
WHERE activity = 'DOORSTEP_DELIVERED'
GROUP BY location
ORDER BY avg_tat_minutes DESC;

-- Stage-by-stage bottleneck heatmap
SELECT prev_activity, activity,
       round(avg(stage_minutes), 2) AS avg_minutes,
       round(100.0 * sum(CASE WHEN sla_breached THEN 1 ELSE 0 END) / count(*), 1) AS sla_breach_pct
FROM bpm.process_events
WHERE stage_minutes IS NOT NULL
GROUP BY prev_activity, activity
ORDER BY sla_breach_pct DESC;

-- Drop-off rate: cases stuck / never delivered
SELECT last_activity, count(*) AS stuck_cases
FROM bpm.case_state
WHERE last_activity <> 'DOORSTEP_DELIVERED'
GROUP BY last_activity
ORDER BY stuck_cases DESC;

-- Active bottleneck: cases currently mid-process, by current stage
SELECT last_activity AS current_stage, count(*) AS cases_in_stage
FROM bpm.case_state
GROUP BY last_activity
ORDER BY cases_in_stage DESC;
```

8. Reproducing These Results

Same Docker stack as the main Data Engineering Lab project — no new services needed for the producer/engine.

```
# 1. Stack already running (docker compose up -d)

# 2. Simulate orders (host terminal)
python bpm/bpm_producer.py --cases 40

# 3. Run the process engine (separate terminal)
docker compose exec spark spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6,\
  io.delta:delta-spark_2.12:3.3.3,org.apache.hadoop:hadoop-aws:3.3.4 \
  bpm/bpm_process_engine.py
```

9. Rubric Mapping (40 Marks)

Component	Marks	Evidence in this report
BPMN diagram, process stages, KPIs, event taxonomy	10	Sections 2–5 — BPMN diagram (Fig. 1), stage table, event schema, KPI table
PySpark SLA & anomaly detection	15	Section 6 — engine architecture, detection rules, and verified live-run results matching every injected anomaly exactly
Querying the lakehouse for process metrics	15	Section 7 — working spark-sql query with verified result today; Trino queries drafted and ready for Week 3

Appendix A: bpm_producer.py

```
"""
bpm_producer.py - simulates a quick-commerce order fulfillment process
(Superstore / Blinkit-Instamart style dark-store dispatch) and publishes
one event per process stage to Kafka topic `order_fulfillment_events`.

Process (happy path):
  ORDER_RECEIVED -> PICKER_ASSIGNED -> [ITEM_SUBSTITUTED]* -> ITEMS_BAGGED
  -> RIDER_ASSIGNED -> DISPATCHED_FROM_DARKSTORE -> DOORSTEP_DELIVERED

Each case (order) is independently, and *deliberately*, given a chance to
carry one or more process anomalies so bpm_process_engine.py has real
things to detect:
  - SLA breach      : one stage takes far longer than its target
  - duplicate task   : the same activity for the same case_id sent twice
  - out-of-sequence : two adjacent stage events swapped in send order
  - substitution     : an ITEM_SUBSTITUTED exception event is inserted
  - drop-off         : the case stops partway through and never delivers
"""

import argparse
import json
import random
import time
from datetime import datetime, timedelta, timezone

from kafka import KafkaProducer

STAGES = [
    "ORDER_RECEIVED",
    "PICKER_ASSIGNED",
    "ITEMS_BAGGED",
    "RIDER_ASSIGNED",
    "DISPATCHED_FROM_DARKSTORE",
    "DOORSTEP_DELIVERED",
]

STAGE_SLA_MINUTES = {
    ("ORDER_RECEIVED", "PICKER_ASSIGNED"): 1,
    ("PICKER_ASSIGNED", "ITEMS_BAGGED"): 3,
    ("ITEMS_BAGGED", "RIDER_ASSIGNED"): 2,
    ("RIDER_ASSIGNED", "DISPATCHED_FROM_DARKSTORE"): 2,
    ("DISPATCHED_FROM_DARKSTORE", "DOORSTEP_DELIVERED"): 7,
}

TARGET_TOTAL_MINUTES = 15
PICKING_SLA_MINUTES = 3

PICKERS = [f"Picker_Emp_{i}" for i in range(101, 121)]
RIDERS = [f"Rider_Emp_{i}" for i in range(201, 231)]
LOCATIONS = [
    "DarkStore_Coimbatore_North",
    "DarkStore_Bangalore_East",
    "DarkStore_Chennai_Central",
    "DarkStore_Hyderabad_West",
    "DarkStore_Pune_South",
]

ITEMS = [
    "Milk 1L", "Bread", "Eggs (6)", "Bananas", "Rice 5kg",
    "Detergent", "Butter 500g", "Toothpaste", "Tea Powder", "Biscuits",
]

ANOMALY_RATES = {
    "sla_breach": 0.15,
    "duplicate": 0.08,
    "out_of_order": 0.08,
    "substitution": 0.10,
    "dropoff": 0.05,
}

def fmt_ts(dt: datetime) -> str:
    return dt.strftime("%Y-%m-%dT%H:%M:%SZ")

def make_event(case_id, activity, ts, resource, location, metadata):
    return {
        "case_id": case_id,
```

```

        "activity": activity,
        "timestamp": fmt_ts(ts),
        "resource": resource,
        "location": location,
        "metadata": metadata,
    }

def simulate_case(case_id, base_ts):
    """Build the full list of (activity, ts, resource, location, metadata) events for one order."""
    location = random.choice(LOCATIONS)
    picker = random.choice(PICKERS)
    rider = random.choice(RIDERS)
    item_count = random.randint(1, 15)
    common_meta = {
        "item_count": item_count,
        "target_total_minutes": TARGET_TOTAL_MINUTES,
        "picking_sla_minutes": PICKING_SLA_MINUTES,
    }

    flags = {name: random.random() < rate for name, rate in ANOMALY_RATES.items()}
    breach_stage = random.choice(list(STAGE_SLA_MINUTES.keys())) if flags["sla_breach"] else None

    events = [("ORDER_RECEIVED", base_ts, "Customer_App", location, dict(common_meta))]
    sim_ts = base_ts
    prev_activity = "ORDER_RECEIVED"
    dropoff_at = random.choice(["ITEMS_BAGGED", "RIDER_ASSIGNED"]) if flags["dropoff"] else None

    for stage in STAGES[1:]:
        sla = STAGE_SLA_MINUTES[(prev_activity, stage)]
        gap = sla + random.uniform(-0.3, -0.02) # clean cases finish comfortably inside the SLA
        if flags["sla_breach"] and (prev_activity, stage) == breach_stage:
            gap = sla + random.uniform(1.5, 4.0)
        sim_ts = sim_ts + timedelta(minutes=gap)

        if stage in ("PICKER_ASSIGNED", "ITEMS_BAGGED"):
            resource = picker
        elif stage in ("RIDER_ASSIGNED", "DISPATCHED_FROM_DARKSTORE"):
            resource = rider
        else:
            resource = "System"

        if flags["substitution"] and stage == "ITEMS_BAGGED":
            sub_ts = sim_ts - timedelta(minutes=random.uniform(0.3, 0.8))
            sub_meta = dict(common_meta)
            sub_meta["substituted_item"] = random.choice(ITEMS)
            sub_meta["reason"] = "out_of_stock"
            events.append(("ITEM_SUBSTITUTED", sub_ts, picker, location, sub_meta))

        events.append((stage, sim_ts, resource, location, dict(common_meta)))
        prev_activity = stage

        if dropoff_at == stage:
            break

    if flags["duplicate"] and len(events) > 2:
        dup_activity, dup_ts, dup_res, dup_loc, dup_meta = random.choice(events[1:])
        events.append((dup_activity, dup_ts + timedelta(seconds=random.uniform(5, 30)), dup_res, dup_loc, dup_meta))

    if flags["out_of_order"] and len(events) > 3:
        # swap only the *timestamps* of two adjacent main-stage events (never touching
        # ITEM_SUBSTITUTED), so the later stage's activity ends up timestamped before the
        # earlier one -- a genuine process-order violation, not just a send-order quirk
        # that gets silently undone once the engine sorts events by event_ts.
        candidates = [
            idx for idx in range(1, len(events) - 1)
            if events[idx][0] != "ITEM_SUBSTITUTED" and events[idx + 1][0] != "ITEM_SUBSTITUTED"
        ]
        if candidates:
            i = random.choice(candidates)
            a_act, a_ts, a_res, a_loc, a_meta = events[i]
            b_act, b_ts, b_res, b_loc, b_meta = events[i + 1]
            events[i] = (a_act, b_ts, a_res, a_loc, a_meta)
            events[i + 1] = (b_act, a_ts, b_res, b_loc, b_meta)

    return events, flags

def main():

```

```

parser = argparse.ArgumentParser(description="Quick-commerce order fulfillment BPM event simulator")
parser.add_argument("--bootstrap", default="localhost:9092", help="Kafka bootstrap servers")
parser.add_argument("--topic", default="order_fulfillment_events")
parser.add_argument("--cases", type=int, default=30, help="number of orders to simulate")
parser.add_argument("--case-interval", type=float, default=1.0, help="real seconds between starting new cases")
parser.add_argument("--event-delay", type=float, default=0.2, help="real seconds between events within one case")
args = parser.parse_args()

producer = KafkaProducer(
    bootstrap_servers=args.bootstrap,
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
)

print(f"Simulating {args.cases} orders -> topic '{args.topic}'. Ctrl+C to stop.")
anomaly_counts = {name: 0 for name in ANOMALY_RATES}

try:
    for n in range(1, args.cases + 1):
        case_id = f"ORD-SUPER-{n:06d}"
        base_ts = datetime.now(timezone.utc)
        events, flags = simulate_case(case_id, base_ts)

        for name, hit in flags.items():
            if hit:
                anomaly_counts[name] += 1
        tags = [name.upper() for name, hit in flags.items() if hit]
        tag_str = f"[{' '.join(tags)}]" if tags else "[clean]"
        print(f"{tag_str} {case_id}: {len(events)} events, last={events[-1][0]}")

        for activity, ts, resource, location, meta in events:
            producer.send(args.topic, value=make_event(case_id, activity, ts, resource, location, meta))
            time.sleep(args.event_delay)

        time.sleep(max(0.0, args.case_interval - len(events) * args.event_delay))
except KeyboardInterrupt:
    print("\nStopping producer.")
finally:
    producer.flush()
    producer.close()
    print(f"Done. Anomaly counts out of {args.cases} cases: {anomaly_counts}")

if __name__ == "__main__":
    main()

```

Appendix B: bpm_process_engine.py

```
"""
```

```
bpm_process_engine.py - PySpark Structured Streaming BPM process-mining engine  
for the quick-commerce order fulfillment case study.
```

```
Reads case events (case_id, activity, timestamp, resource, location, metadata)  
from Kafka topic `order_fulfillment_events` and, per micro-batch:
```

1. Reconstructs each case's process state (in-batch via window functions, across batches via a persisted `case\_state` Delta table), so multi-event cases are handled correctly even when several of a case's events land in the same micro-batch.
2. Flags four kinds of process anomalies on every event:
 - `is_duplicate` : same (case_id, activity) observed twice
 - `is_out_of_sequence` : activity arrived out of the expected stage order
 - `sla_breached` : this stage-to-stage gap exceeded its SLA
 - `total_sla_breached` : (at DOORSTEP_DELIVERED) total case time exceeded `metadata.target_total_minutes`
3. Writes every event, enriched with the above, to a Delta table ``bpm.process_events`` (the immutable audit / fact log) and upserts the latest per-case snapshot into ``bpm.case_state`` (a Delta MERGE).
4. Registers both tables in Hive Metastore (once) and prints a running KPI summary each batch: SLA breach rate, duplicate/out-of-sequence counts, per-stage average duration (bottleneck signal), and throughput.

```
Run inside the `spark` container:
```

```
docker compose exec spark spark-submit \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6,io.delta:delta-spark_2.12:3.3.3,org.apache.hadoop:hadoop-  
bpm/bpm_process_engine.py
```

```
"""
```

```
import argparse  
import os
```

```
from pyspark.sql import SparkSession, Window, functions as F  
from pyspark.sql.types import (  
    StructType, StructField, StringType, IntegerType, TimestampType, DoubleType  
)
```

```
STAGE_INDEX = {  
    "ORDER_RECEIVED": 0,  
    "PICKER_ASSIGNED": 1,  
    "ITEMS_BAGGED": 2,  
    "RIDER_ASSIGNED": 3,  
    "DISPATCHED_FROM_DARKSTORE": 4,  
    "DOORSTEP_DELIVERED": 5,  
}
```

```
# None => data-driven from metadata.picking_sla_minutes for that specific transition;  
# the assignment's example payload only ships a picking-stage SLA, so the other  
# stage targets are fixed ops constants (documented in bpm/README.md).
```

```
STAGE_SLA_MINUTES = {  
    ("ORDER_RECEIVED", "PICKER_ASSIGNED"): 1.0,  
    ("PICKER_ASSIGNED", "ITEMS_BAGGED"): None,  
    ("ITEMS_BAGGED", "RIDER_ASSIGNED"): 2.0,  
    ("RIDER_ASSIGNED", "DISPATCHED_FROM_DARKSTORE"): 2.0,  
    ("DISPATCHED_FROM_DARKSTORE", "DOORSTEP_DELIVERED"): 7.0,  
}
```

```
EVENT_SCHEMA = StructType([  
    StructField("case_id", StringType(), True),  
    StructField("activity", StringType(), True),  
    StructField("timestamp", StringType(), True),  
    StructField("resource", StringType(), True),  
    StructField("location", StringType(), True),  
    StructField("metadata", StructType([  
        StructField("item_count", IntegerType(), True),  
        StructField("target_total_minutes", IntegerType(), True),  
        StructField("picking_sla_minutes", IntegerType(), True),  
        StructField("substituted_item", StringType(), True),  
        StructField("reason", StringType(), True),  
    ]), True),  
)
```

```
STATE_SCHEMA = StructType([  
    StructField("case_id", StringType(), False),  
    StructField("last_activity", StringType(), True),
```

```

    StructField("last_activity_ts", TimestampType(), True),
    StructField("last_stage_index", IntegerType(), True),
    StructField("order_received_ts", TimestampType(), True),
    StructField("item_count", IntegerType(), True),
    StructField("target_total_minutes", IntegerType(), True),
    StructField("picking_sla_minutes", IntegerType(), True),
    StructField("updated_at", TimestampType(), True),
])

def stage_index_col(activity_col):
    expr = F.lit(None).cast("int")
    for name, idx in STAGE_INDEX.items():
        expr = F.when(activity_col == name, F.lit(idx)).otherwise(expr)
    return expr

def sla_limit_minutes_col(prev_activity_col, activity_col, picking_sla_col):
    expr = F.lit(None).cast("double")
    for (prev, cur), minutes in STAGE_SLA_MINUTES.items():
        cond = (prev_activity_col == prev) & (activity_col == cur)
        value = picking_sla_col.cast("double") if minutes is None else F.lit(minutes)
        expr = F.when(cond, value).otherwise(expr)
    return expr

class RunningTotals:
    def __init__(self):
        self.events = 0
        self.cases_delivered = 0
        self.sla_breaches = 0
        self.duplicates = 0
        self.out_of_sequence = 0
        self.substitutions = 0
        self.process_registered = False
        self.state_registered = False

def load_state(spark, path):
    try:
        return spark.read.format("delta").load(path)
    except Exception:
        return spark.createDataFrame([], STATE_SCHEMA)

def make_batch_processor(spark, process_events_path, case_state_path, database, totals: RunningTotals):
    def process_batch(batch_df, batch_id: int):
        batch_df = (
            batch_df.withColumn("event_ts", F.to_timestamp("timestamp"))
            .withColumn("stage_index", stage_index_col(F.col("activity")))
            .cache()
        )

        sub_df = batch_df.filter(F.col("activity") == "ITEM_SUBSTITUTED")
        main_df = batch_df.filter(F.col("activity") != "ITEM_SUBSTITUTED")

        if main_df.rdd.isEmpty() and sub_df.rdd.isEmpty():
            batch_df.unpersist()
            return

        state_df = load_state(spark, case_state_path).select(
            F.col("case_id").alias("st_case_id"),
            F.col("last_activity").alias("st_last_activity"),
            F.col("last_activity_ts").alias("st_last_activity_ts"),
            F.col("last_stage_index").alias("st_last_stage_index"),
            F.col("order_received_ts").alias("st_order_received_ts"),
        )

        w = Window.partitionBy("case_id").orderBy("event_ts")
        main_df = (
            main_df.withColumn("in_batch_prev_activity", F.lag("activity").over(w))
            .withColumn("in_batch_prev_ts", F.lag("event_ts").over(w))
            .withColumn("in_batch_prev_idx", F.lag("stage_index").over(w))
            .withColumn(
                "in_batch_order_ts",
                F.min(F.when(F.col("activity") == "ORDER_RECEIVED", F.col("event_ts"))).over(
                    Window.partitionBy("case_id")
                )
            ),
        )
    
```

```

)

joined = main_df.join(state_df, main_df.case_id == state_df.st_case_id, "left")

enriched = (
    joined.withColumn(
        "prev_activity", F.coalesce(F.col("in_batch_prev_activity"), F.col("st_last_activity"))
    )
    .withColumn("prev_ts", F.coalesce(F.col("in_batch_prev_ts"), F.col("st_last_activity_ts")))
    .withColumn("prev_stage_index", F.coalesce(F.col("in_batch_prev_idx"), F.col("st_last_stage_index")))
    .withColumn(
        "order_received_ts", F.coalesce(F.col("in_batch_order_ts"), F.col("st_order_received_ts"))
    )
    .withColumn(
        "stage_minutes",
        F.when(
            F.col("prev_ts").isNotNull(),
            (F.col("event_ts").cast("long") - F.col("prev_ts").cast("long")) / 60.0,
        ),
    )
    .withColumn(
        "is_duplicate",
        (F.col("activity") == F.col("prev_activity")).cast("boolean"),
    )
    .withColumn(
        "is_out_of_sequence",
        F.when(F.col("prev_stage_index").isNull(), F.col("stage_index") != 0)
        .when(F.col("activity") == F.col("prev_activity"), F.lit(False))
        .otherwise(F.col("stage_index") <= F.col("prev_stage_index")),
    )
    .withColumn(
        "sla_limit_minutes",
        sla_limit_minutes_col(
            F.col("prev_activity"), F.col("activity"), F.col("metadata.picking_sla_minutes")
        ),
    )
    .withColumn(
        "sla_breached",
        F.when(
            F.col("sla_limit_minutes").isNotNull() & F.col("stage_minutes").isNotNull(),
            F.col("stage_minutes") > F.col("sla_limit_minutes"),
        ).otherwise(F.lit(False)),
    )
    .withColumn(
        "total_minutes",
        F.when(
            F.col("order_received_ts").isNotNull(),
            (F.col("event_ts").cast("long") - F.col("order_received_ts").cast("long")) / 60.0,
        ),
    )
    .withColumn(
        "total_sla_breached",
        (F.col("activity") == "DOORSTEP_DELIVERED")
        & F.col("total_minutes").isNotNull()
        & (F.col("total_minutes") > F.col("metadata.target_total_minutes")),
    )
)

sub_enriched = (
    sub_df.withColumn("prev_activity", F.lit(None).cast("string"))
    .withColumn("stage_minutes", F.lit(None).cast("double"))
    .withColumn("is_duplicate", F.lit(False))
    .withColumn("is_out_of_sequence", F.lit(False))
    .withColumn("sla_limit_minutes", F.lit(None).cast("double"))
    .withColumn("sla_breached", F.lit(False))
    .withColumn("total_minutes", F.lit(None).cast("double"))
    .withColumn("total_sla_breached", F.lit(False))
)

fact_cols = [
    "case_id", "activity", "event_ts", "resource", "location", "metadata",
    "prev_activity", "stage_minutes", "is_duplicate", "is_out_of_sequence",
    "sla_limit_minutes", "sla_breached", "total_minutes", "total_sla_breached",
]
fact_df = enriched.select(*fact_cols).unionByName(sub_enriched.select(*fact_cols))
fact_df = fact_df.withColumn("event_date", F.to_date("event_ts")).cache()

fact_count = fact_df.count()
if fact_count > 0:

```

```

fact_df.write.format("delta").mode("append").partitionBy("event_date").save(process_events_path)
if not totals.process_registered:
    spark.sql(f"CREATE DATABASE IF NOT EXISTS {database}")
    spark.sql(
        f"CREATE TABLE IF NOT EXISTS {database}.process_events USING DELTA LOCATION '{process_events_path}'"
    )
    totals.process_registered = True

# latest observed state per case in this batch (audit log keeps every event;
# this projection is just "what we last saw", warts and all)
latest_w = Window.partitionBy("case_id").orderBy(F.col("event_ts").desc())
new_state = (
    enriched.withColumn("rn", F.row_number().over(latest_w))
    .filter(F.col("rn") == 1)
    .select(
        "case_id",
        F.col("activity").alias("last_activity"),
        F.col("event_ts").alias("last_activity_ts"),
        F.col("stage_index").alias("last_stage_index"),
        F.col("order_received_ts"),
        F.col("metadata.item_count").alias("item_count"),
        F.col("metadata.target_total_minutes").alias("target_total_minutes"),
        F.col("metadata.picking_sla_minutes").alias("picking_sla_minutes"),
        F.current_timestamp().alias("updated_at"),
    )
)

if new_state.rdd.isEmpty():
    pass
elif totals.state_registered or _delta_table_exists(spark, case_state_path):
    from delta.tables import DeltaTable

    target = DeltaTable.forPath(spark, case_state_path)
    (
        target.alias("t")
        .merge(new_state.alias("s"), "t.case_id = s.case_id")
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
else:
    new_state.write.format("delta").mode("overwrite").save(case_state_path)

if not totals.state_registered and not new_state.rdd.isEmpty():
    spark.sql(f"CREATE DATABASE IF NOT EXISTS {database}")
    spark.sql(
        f"CREATE TABLE IF NOT EXISTS {database}.case_state USING DELTA LOCATION '{case_state_path}'"
    )
    totals.state_registered = True

# ---- console KPI report for this batch ----
sla_breach_count = enriched.filter(F.col("sla_breached") | F.col("total_sla_breached")).count()
dup_count = enriched.filter(F.col("is_duplicate")).count()
oos_count = enriched.filter(F.col("is_out_of_sequence")).count()
sub_count = sub_df.count()
delivered_count = enriched.filter(F.col("activity") == "DOORSTEP_DELIVERED").count()

totals.events += fact_count
totals.cases_delivered += delivered_count
totals.sla_breaches += sla_breach_count
totals.duplicates += dup_count
totals.out_of_sequence += oos_count
totals.substitutions += sub_count

print(f"\n==== batch {batch_id}: {fact_count} event(s) =====")
if fact_count > 0:
    enriched.select(
        "case_id", "activity", "prev_activity", F.round("stage_minutes", 2).alias("stage_minutes"),
        "sla_limit_minutes", "sla_breached", "is_duplicate", "is_out_of_sequence", "total_sla_breached",
    ).show(20, truncate=False)

bottleneck = (
    enriched.filter(F.col("stage_minutes").isNotNull())
    .groupBy("prev_activity", "activity")
    .agg(F.round(F.avg("stage_minutes"), 2).alias("avg_minutes"), F.count("*").alias("n"))
    .orderBy(F.desc("avg_minutes"))
)
if bottleneck.count() > 0:
    print("---- stage durations this batch (bottleneck candidates) ----")

```

```

        bottleneck.show(10, truncate=False)

    print(
        f">>> batch KPIs – sla_breaches: {sla_breach_count} | duplicates: {dup_count} | "
        f"out_of_sequence: {oos_count} | substitutions: {sub_count} | delivered (throughput): {delivered_count}"
    )
    print(
        f">>> RUNNING TOTALS – events: {totals.events} | delivered: {totals.cases_delivered} | "
        f"sla_breaches: {totals.sla_breaches} | duplicates: {totals.duplicates} | "
        f"out_of_sequence: {totals.out_of_sequence} | substitutions: {totals.substitutions}\n"
    )

    batch_df.unpersist()
    fact_df.unpersist()

    return process_batch

def _delta_table_exists(spark, path):
    try:
        from delta.tables import DeltaTable

        return DeltaTable.isDeltaTable(spark, path)
    except Exception:
        return False

def build_spark_session(args):
    return (
        SparkSession.builder.appName("bpm-order-fulfillment-engine")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .config("spark.sql.warehouse.dir", "file:///warehouse")
        .config("spark.hadoop.fs.s3a.endpoint", args.s3_endpoint)
        .config("spark.hadoop.fs.s3a.access.key", args.s3_access_key)
        .config("spark.hadoop.fs.s3a.secret.key", args.s3_secret_key)
        .config("spark.hadoop.fs.s3a.path.style.access", "true")
        .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
        .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
        .config("spark.sql.catalogImplementation", "hive")
        .config("spark.sql.hive.metastore.jars", "builtin")
        .enableHiveSupport()
        .getOrCreate()
    )

def main():
    parser = argparse.ArgumentParser(description="BPM order-fulfillment SLA + anomaly detection engine")
    parser.add_argument("--bootstrap", default="kafka:19092")
    parser.add_argument("--topic", default="order_fulfillment_events")
    parser.add_argument("--starting-offsets", default="earliest", choices=["earliest", "latest"])
    parser.add_argument("--process-events-path", default="s3a://lakehouse/bpm/process_events")
    parser.add_argument("--case-state-path", default="s3a://lakehouse/bpm/case_state")
    parser.add_argument("--checkpoint-path", default="s3a://lakehouse/checkpoints/bpm_process_engine")
    parser.add_argument("--database", default="bpm")
    parser.add_argument("--trigger-seconds", type=int, default=10)
    parser.add_argument("--s3-endpoint", default="http://minio:9000")
    parser.add_argument("--s3-access-key", default=os.environ.get("MINIO_ACCESS_KEY", "minioadmin"))
    parser.add_argument("--s3-secret-key", default=os.environ.get("MINIO_SECRET_KEY", "minioadmin123"))
    args = parser.parse_args()

    spark = build_spark_session(args)
    spark.sparkContext.setLogLevel("WARN")

    raw = (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", args.bootstrap)
        .option("subscribe", args.topic)
        .option("startingOffsets", args.starting_offsets)
        .option("failOnDataLoss", "false")
        .load()
    )

    parsed = raw.select(
        F.from_json(F.col("value").cast("string"), EVENT_SCHEMA).alias("data")
    ).select("data.*")

    totals = RunningTotals()
    query = (

```

```
        parsed.writeStream.foreachBatch(  
            make_batch_processor(spark, args.process_events_path, args.case_state_path, args.database, totals)  
        )  
        .option("checkpointLocation", args.checkpoint_path)  
        .trigger(processingTime=f"{args.trigger_seconds} seconds")  
        .start()  
    )  
  
    query.awaitTermination()  
  
if __name__ == "__main__":  
    main()
```